

SMART 64-Bit ID — Technical Paper

Technical Specification: v1.4

Date: 2026-10-04

Copyright: © 2026 MD. AZIZUR RAHMAN

Organization: SAMARA

Abstract

SMART 64-Bit ID is a fixed-width 64-bit identifier model designed to provide a compact immutable identity core together with bounded governance and lifecycle metadata. The identifier contains a 50-bit identity core composed of a 29-bit Local ID and a 21-bit Engine ID, plus 14 bits reserved for Region, State, Reserved, and Version semantics.

The design separates identity from governance metadata. Engine and Local ID form the immutable identity core and support deterministic fixed-position routing. Region is generation-time metadata rather than a routing selector. State represents lifecycle status and is not a routing selector. Reserved and Version fields provide bounded future governance and format evolution.

SMART ID requires authoritative Engine ownership, persistent Local ID allocation, non-reuse, hard failure on exhaustion or uniqueness violations, and crash-safe allocation behavior. The specification intentionally does not mandate a particular control-plane topology, database vendor, deployment architecture, authentication framework, or cryptographic implementation.

Empirical testing documented in the project research material shows approximate single-engine performance parity with the tested traditional identifier baseline and a 33–53% smaller clustered index across the tested MariaDB/InnoDB configurations. These storage findings are workload- and environment-specific. A separate simulated multi-engine experiment is explicitly invalid as evidence of distributed multi-engine performance.

SMART ID is intended to be implemented with appropriate application security controls and, where format-preserving encryption is used for public representations, with applicable internationally recognized and approved cryptographic standards and their current authoritative revisions.

1. Introduction

Distributed systems frequently need identifiers that remain unique across multiple allocation domains while supporting efficient storage and deterministic routing. SMART 64-Bit ID addresses this problem by dividing a fixed 64-bit value into an immutable identity core and a governance/metadata region.

The model is deliberately narrow. It defines identifier semantics, allocation correctness, routing semantics, lifecycle behavior, persistence requirements, and required failure behavior. It does not attempt to define an entire distributed database, service mesh, authentication system, or control-plane product.

The technical baseline in this paper is locked to SMART ID Technical Specification v1.4.

2. 64-Bit Layout

The identifier is defined from least-significant bit to most-significant bit as follows:

Field	Bits	Width	Semantics
Local ID	0–28	29	Immutable identity within an Engine
Engine	29–49	21	Immutable allocation/routing domain
Region	50–57	8	Generation-time metadata
State	58	1	Lifecycle marker
Reserved	59–62	4	Reserved for future governance
Version	63	1	Format version marker

The identity core is:

$$\text{Engine} + \text{Local ID} = 21 + 29 = 50 \text{ bits}$$

The governance/metadata portion is:

$$\text{Region} + \text{State} + \text{Reserved} + \text{Version} = 8 + 1 + 4 + 1 = 14 \text{ bits}$$

The v1.4 layout is fixed and must not be reinterpreted by implementations.

3. Identity Capacity

3.1 Local ID

The Local ID occupies 29 bits:

$$2^{29} = 536,870,912 \text{ identifiers per Engine.}$$

Local IDs are allocated monotonically within an Engine namespace. Allocation state must persist across restarts and must never wrap or reuse previously allocated values.

A randomized non-zero starting point may be used, but it consumes part of the finite Local ID range and therefore must be treated as an explicit capacity trade-off.

When the Local ID namespace is exhausted, generation must **FAIL HARD**. An implementation must not wrap, silently reuse, or reinterpret the namespace.

3.2 Engine

The Engine field occupies 21 bits:

$$2^{21} = 2,097,152 \text{ possible Engine IDs.}$$

Engine assignment is immutable for an already-issued SMART ID.

At any time, an Engine ID MUST have at most one authoritative active allocator. Concurrent or stale ownership of the same Engine ID MUST result in generation failure until ownership is unambiguously established.

Fencing, leases, epochs, or equivalent ownership mechanisms are implementation choices. SMART ID does not mandate a particular control-plane architecture.

4. Generation Semantics

A conforming generation process must establish authoritative Engine ownership before generating identifiers.

A conceptual generation sequence is:

1. Establish authoritative ownership of an Engine.
2. Obtain the next persistent Local ID from that Engine.
3. Validate that the Local ID has not previously been used.
4. Assemble the 64-bit SMART ID.
5. Persist the identifier according to the application's transactional requirements.
6. Commit the operation.
7. Return the identifier.

Concurrency control is required wherever allocation state may be shared. Row-level locking or an equivalent mechanism may be used.

Crash recovery must never reuse an allocated Local ID. Gaps caused by failed transactions, crashes, or abandoned allocations are acceptable. Uniqueness and non-reuse take priority over gaplessness.

Batch allocation may be used. Implementations should document whether ordering describes reservation order, allocation order, commit order, or another observable sequence.

5. Uniqueness and Failure Rules

The core uniqueness invariants are:

- No duplicate authoritative Engine ownership.
- No duplicate Local ID within an Engine namespace.
- No reuse of an already allocated SMART ID.
- No silent Local ID wrap.
- No continuation after Local ID exhaustion.
- No generation while Engine ownership is ambiguous.

A violation of the identifier's uniqueness or ownership invariants must **FAIL HARD** rather than silently producing potentially conflicting identifiers.

An implementation may choose its own operational response after a hard failure, such as obtaining another Engine, restoring authoritative ownership, or stopping the affected generation service. SMART ID defines the correctness requirement, not the operational topology.

6. Routing

The intended routing path is:

64-bit SMART ID → extract Engine → route to Engine → extract Local ID → full primary-key lookup

Engine is the routing selector.

Local ID identifies the record within the Engine namespace.

Region is not part of the routing decision.

State is not a routing selector. It may be used as an index-level lifecycle filter.

Reserved and Version bits do not replace the Engine routing field.

Because Engine occupies a fixed bit position, extraction is constant with respect to the identifier's fixed width. No universal CPU-cycle count is claimed; actual instruction count, CPU cycles, cache behavior, storage latency, and end-to-end performance depend on implementation and environment.

7. Lifecycle and State

State is a lifecycle marker rather than an identity component.

For v1.4:

- **State = 1:** enabled
- **State = 0:** retired/disabled

State must not be used to change the identity of an existing identifier.

If an identity is transferred, replaced, or otherwise requires a new identifier, the implementation creates a **new SMART ID** and retires the old SMART ID. The old identifier is never reused.

This preserves immutable identity while allowing lifecycle governance.

8. Region and Governance Metadata

Region is an 8-bit metadata field representing a generation-time region or country classification as defined by the implementation's governance policy.

Region is not identity and is not a routing selector.

Implementations may remap Region for presentation or governance purposes. Historical interpretation may preserve the generation-time value as a snapshot.

Reserved bits are currently zero in v1.4 unless a future specification explicitly defines another meaning.

Version 0 represents the v1.x format family. Version 1 is reserved for a future v2 format.

9. Control Plane and Engine Ownership

SMART ID does not mandate a particular Engine allocation or control-plane architecture.

A reference implementation may use a centralized control application because it is practical, but that is an implementation choice rather than a SMART ID requirement.

The client is responsible for ensuring authoritative and non-conflicting Engine ownership.

At any time, an Engine ID **MUST** have at most one authoritative active allocator. Stale or concurrent ownership, including ownership left ambiguous after failure or network partition, must not be allowed to produce identifiers.

Fencing or lease ownership is an appropriate implementation technique, but other mechanisms may satisfy the same invariant.

10. Persistence and Crash Recovery

Local allocation state must be durable enough to prevent reuse after restart or failure.

The exact storage mechanism is an implementation choice. SQLite, MyISAM, or another mechanism without appropriate concurrency and durability guarantees is outside the intended correctness model for shared allocation state.

A crash may create unused gaps. Such gaps do not violate SMART ID semantics.

The fundamental requirement is:

Allocated identifiers must never be reused.

11. Public Representation, FPE, and Security

SMART internal identity and routing are separate from public-ID cryptographic processing.

An implementation may maintain an internal database primary key and expose a SMART ID as a public identifier. If a public representation requires format-preserving encryption (FPE), that cryptographic layer is separate from the SMART ID bit-extraction and routing model.

SMART ID implementations **MUST** use format-preserving encryption and related cryptographic mechanisms in accordance with applicable internationally recognized and approved cryptographic standards and their current authoritative revisions.

SMART ID does not define or require a proprietary cryptographic algorithm.

FPE is not authentication, authorization, or integrity protection. Application and deployment owners remain responsible for authentication, authorization, key management, access control, integrity, logging, monitoring, and other security controls.

SMART ID itself does not claim cryptographic certification.

12. Serialization and APIs

SMART IDs should be serialized as strings in JSON and similar interchange formats.

Implementations should not rely on native numeric parsing for values above the exact integer range supported by the target language/runtime. In particular, common IEEE-754-based JavaScript numeric handling does not exactly represent every integer above 2^{53} .

String serialization avoids ambiguity and preserves the complete 64-bit value.

13. Database Integration

SMART ID can be used as a database primary key or as a separate public identifier associated with another internal key.

The choice depends on the application’s schema and operational requirements.

The design is compatible with relational databases and other storage systems capable of representing the complete 64-bit value and enforcing the required uniqueness semantics.

The SMART specification does not mandate a database vendor, index implementation, storage engine, shard technology, or deployment topology.

14. Empirical Single-Engine Findings

The valid v1.4 experiment used:

- N = 1,000,000 rows
- MariaDB 10.4.32
- InnoDB
- Linux
- ASUS ZenBook UX363JA
- local NVMe SSD
- one database instance
- no competing workload

The tested identifier configurations compared a traditional baseline with SMART ID under the same workload.

14.1 Insert/load performance

Measured results:

Buffer pool	Traditional	SMART ID	Difference
512M	16.68 s	15.96 s	-4%
64M	18.52 s	20.40 s	+10%
16M	13.70 s	14.86 s	+8%

Interpretation: the measured results are consistent with approximate performance parity in the tested single-engine environment. They do not establish a universal latency advantage or disadvantage.

14.2 Point lookup

For 10,000 point lookups:

Buffer pool	Traditional	SMART ID	Difference
512M	2.951 s	2.990 s	+1.3%
64M	3.201 s	3.107 s	-2.9%
16M	3.148 s	3.194 s	+1.5%

The measured difference remained within approximately $\pm 3\%$.

14.3 Range scan

For 1,000 scans of 1,000-row ranges:

Buffer pool	Traditional	SMART ID	Difference
512M	15.584 s	15.515 s	-0.4%
64M	15.624 s	15.658 s	+0.2%
16M	15.679 s	15.970 s	+1.9%

The measured difference remained within approximately $\pm 2\%$.

14.4 Clustered index size

Measured clustered index sizes:

Buffer pool	Traditional	SMART ID	SMART reduction
512M	69.84 MB	42.55 MB	39%
64M	69.84 MB	33.10 MB	53%
16M	69.84 MB	46.74 MB	33%

The tested configurations therefore showed a **33–53% smaller clustered index** for SMART ID.

This result is specific to the tested MariaDB/InnoDB workload and configuration. It is not a universal storage guarantee.

The observed effect is associated with the tested SMART ID ordering and the resulting distribution of key bytes in the clustered index. Storage-engine behavior, schema, workload, data distribution, and configuration can change the result.

15. Multi-Engine Research Boundary

A separate experiment attempted to represent multiple Engines by cycling Engine values in a single writer process and single connection.

That experiment is explicitly **INVALID as evidence of multi-engine distributed performance**.

It did not represent:

- 16 independent authoritative Engines;
- 16 concurrent writers;
- independent allocation state;
- independent connections;
- independent commit streams; or
- genuine distributed ownership.

Therefore, its timing and storage results must not be used to claim multi-engine scalability.

A valid future multi-engine experiment should use independent concurrent writers, each with authoritative ownership of its own Engine, or otherwise model genuine independent allocation domains.

16. Limitations

SMART ID does not guarantee:

- universal performance improvement;
- universal storage reduction;
- a specific CPU-cycle count;
- a specific database index size;
- distributed scalability without appropriate implementation;
- authentication or authorization;
- cryptographic certification;
- protection against application-level misuse; or
- a particular control-plane architecture.

The benchmark results are empirical observations from defined environments, not universal guarantees.

FPE performance is separate from SMART ID bit extraction performance.

The implementation owner must validate correctness, performance, security, durability, and operational behavior in the actual deployment environment.

17. Architectural Boundary

SMART ID defines

- the 64-bit structure;
- identity semantics;
- Engine and Local ID allocation rules;
- uniqueness invariants;
- routing semantics;
- lifecycle semantics;
- required failure behavior;
- persistence/non-reuse requirements; and
- the cryptographic standards boundary.

SMART ID does not dictate

- client control-plane topology;
- Engine provisioning architecture;
- lease or fencing technology;
- database vendor;
- storage-engine configuration;
- deployment topology;
- authentication system;
- authorization system;
- key-management architecture; or
- a proprietary cryptographic algorithm.

This separation is intentional. SMART ID defines the identifier contract while allowing implementations to choose appropriate infrastructure.

18. Implementation Checklist

A conforming implementation should verify at minimum:

1. The complete 64-bit value is preserved.
2. Engine extraction uses bits 29–49.
3. Local ID occupies bits 0–28.
4. Engine ownership is authoritative and non-conflicting.
5. At most one active allocator controls an Engine.
6. Local allocation state persists across restart.

7. Allocated Local IDs are never reused.
8. Local ID exhaustion fails hard.
9. Duplicate Engine ownership fails hard.
10. Duplicate Local ID generation fails hard.
11. State is treated as lifecycle metadata, not routing.
12. Region is treated as metadata, not routing.
13. FPE is kept separate from internal routing semantics.
14. Public API serialization preserves the full 64-bit value.
15. Security controls are implemented outside the identifier definition.
16. Performance claims are supported by reproducible measurements.
17. Multi-engine claims use genuine independent allocation domains.

19. Conclusion

SMART 64-Bit ID v1.4 defines a compact 64-bit identifier with a 50-bit immutable identity core and 14 bits of governance and metadata.

Its central design principles are:

- immutable identity;
- authoritative Engine ownership;
- persistent monotonic Local allocation;
- non-reuse;
- hard failure on correctness violations;
- deterministic Engine-based routing;
- explicit lifecycle semantics;
- separation of identity from governance;
- separation of internal routing from public cryptographic processing; and
- bounded, reproducible technical claims.

The valid single-engine measurements demonstrate approximate performance parity in the tested environment and a 33–53% clustered-index size reduction across the tested MariaDB/InnoDB configurations. Those measurements are workload-specific and should not be generalized beyond their experimental boundary.

The multi-engine experiment documented by the project is intentionally classified as invalid evidence for distributed performance. Future distributed testing must use genuine independent authoritative Engines and concurrent writers.

SMART ID v1.4 therefore establishes a precise identifier contract while leaving infrastructure and deployment choices to implementations.

Correctness first. Reproducibility second. Performance claims third.

Status

SMART ID Technical Specification v1.4 is locked for this release.

This paper is a technical description of the v1.4 model and does not modify the normative specification.

**Copyright (c) 2026 MD. AZIZUR RAHMAN
SAMARA**